SMART HOME CONTROL SYSTEM SIMULATOR

BY IVAN V. TUKALO
SUPERVISOR: ANDRII V. YARMILKO

CONTENTS

REASON AND PURPOSE FOR CREATING THE SIMULATOR

The modern dynamic pace of life requires living spaces to provide comfort and safety, which are fundamental to human productivity. These basic characteristics of the environment are directly related to the fundamental needs reflected in Maslow's pyramid, such as physiological comfort and the need for protection. Automated control of temperature, humidity, and lighting is becoming a necessary condition for creating a healthy and happy living environment.

Modern smart home systems integrate various devices in a living space to provide comfort, safety, and energy efficiency, but designing the logic of their interaction remains a complex engineering task. The process of implementing such technologies requires preliminary research of operating scenarios and possible equipment configurations. These circumstances create a need for specialized software simulators capable of effectively simulating the operation of automated systems.

The purpose of this coursework is to design and implement a smart home control system simulator using object-oriented programming principles and to gain practical skills in software system development.

TASKS

1. Analyze the subject area, research existing sources, and review existing smart home software products to identify key approaches and algorithms.
2. Formulate the task: detail the requirements for the simulator's functionality, describe its future functions and business logic.
3. Design the architecture of the software product. Design an object-oriented model of the system: develop a class hierarchy for smart devices, applying the principles of inheritance, encapsulation, and polymorphism.
4. Implement the developed simulator in a high-level language and create a graphical user interface.
5. Describe key software solutions and code fragments, build a class diagram of the realized system, and test the functionality by simulating typical scenarios.

SELECTED APPROACHES AND TOOLS

A fixed-step algorithm using a timer running in the interface thread was chosen as the simulation runner. This approach is more suitable than an event-driven one for this task, as it provides the necessary continuous visualization, smooth time flow, and allows correct modeling of continuous processes, such as gradual changes in temperature or humidity in rooms.

To build the graphical interface, I used an approach based on a declarative framework instead of traditional imperative libraries. This decision was made due to the existence of powerful tools for working with vector graphics and the Canvas container, which allows for easy realization of a 2D house plan with interactive elements overlaying it, as well as support for a mechanism for clear separation of logic and visual representation.

The logical core is built on a decentralized algorithm using polymorphism instead of creating a single large control class. This has made it possible to create a flexible hierarchy where each device independently controls its behavior through the realization of an abstract state update method, which greatly simplifies system scaling and adherence to encapsulation principles without the need to change the main program loop when adding new device types.

To link business logic and visualization, I used the Binding algorithm for automatic data binding instead of manually updating interface elements. Thanks to this approach and the realization of the notification interface, any modification of the logical state of the device in the code is instantly displayed on the screen without writing additional code for redrawing, which guarantees the synchronization of data and the interface.

The C# programming language and the .NET platform were chosen for the software implementation of the simulator. This choice was made due to the presence of strict typing and advanced capabilities for object-oriented programming, which is critical for building a stable architecture. Another important factor was the availability of powerful LINQ libraries, which allow efficient and compact processing of data collections and lists of active devices.

Windows Presentation Foundation is used as the technology for building the graphical interface. Unlike outdated technologies such as WinForms, WPF provides the ability to use vector graphics and Canvas containers to precisely place objects on top of the plan using absolute coordinates. In addition, the use of the declarative XAML markup language provides a clear architectural separation between the visual description of the interface and the program logic.

Microsoft Visual Studio was chosen as the development environment. This toolkit provides the most convenient tools for working with the selected technology stack, including a built-in XAML designer for visual interface design and powerful tools for code debugging. The refactoring tools of this environment allow you to effectively maintain code purity and the speed of development of a complex project.

```
public abstract class SmartDeviceBase : INotifyPropertyChanged
{
    private DeviceState _currentState;

    public DeviceState CurrentState
    {
        get => _currentState;
        protected set
        {
            if (_currentState != value)
            {
                _currentState = value;
                OnPropertyChanged();
            }
        }
    }
}
```

USE OF ENCAPSULATION

Encapsulation is used in the `SmartDeviceBase` base class for the `CurrentState` property, where the setter is declared with the protected access modifier. This approach makes it impossible to change the state of the device from external classes, leaving this right exclusively to the device itself or its descendants. This ensures the reliability of the system, as it guarantees that switching between operating modes will only occur through the provided internal algorithms.

```
public class SelectedRoomInfoViewModel : INotifyPropertyChanged
{
    private Room _currentRoom; // Private field

    public void SelectRoom(Room room)
    {
        _currentRoom = room;
        IsRoomSelected = true;

        // Internal update logic, hidden from the outside
        DevicesInRoom.Clear();
        UpdateThermostats();
        RefreshProperties();
    }
}
```

Encapsulation is used in the `SelectedRoomInfoViewModel` class by closing direct access to the `_currentRoom` field and realization the `SelectRoom` method to change the active room. This hides all the complex logic of updating the interface, including clearing device lists and searching for thermostats, inside the class. This allows external code to simply call a single method without worrying about the internal implementation of data updates.

```
public class BatteryDevice : SmartDeviceBase
{
    // External code can only read the charge level
    public double ChargeLevel { get; private set; } = 100.0;

    public override void UpdateState(...)
    {
        // Charge changes only occur within the class
        if (IsCharging)
            ChargeLevel = Math.Min(100.0, ChargeLevel +
CHARGE_PER_MIN);
    }
}
```

Encapsulation is used in the `BatteryDevice` class for the `ChargeLevel` property, which has a private setter `private set`. This prevents "magical" changes to the charge level from outside the program, forcing any changes to the energy level to go exclusively through the `UpdateState` method. This solution guarantees that the battery will only discharge or charge according to the rules of the physical time model.

```
public partial class MainWindow : Window
{
    // Private fields, not available to other classes
    private DispatcherTimer _simulationTimer;
    private TimeSpan _currentSimTime;

    // Public property for safe access with change notification
    public TimeSpan CurrentSimTime
    {
        get => _currentSimTime;
        set
        {
            _currentSimTime = value;
            OnPropertyChanged();
        }
    }
}
```

Encapsulation is used in the `MainWindow` class to isolate the logic of the simulation timer `_simulationTimer` and the current time field `_currentSimTime`. Since the timer is a private field, no other component of the system can accidentally stop or change the speed of time. Access to time changes is only provided through controlled event handling methods of the interface.

```
public abstract class SmartDeviceBase : INotifyPropertyChanged
{
    public string Name { get; }

    // An abstract method defines "what" a device should do,
    // but not "how" it does it. Implementation is left to
inheritors.
    public abstract void UpdateState(TimeSpan currentTime, bool
isElectricityOn, Room environment);
}
```

USE OF ABSTRACTION

Abstraction is used in the design of the base class `SmartDeviceBase`, in which the abstract method `UpdateState` is declared without realization. This allows the main simulation controller to interact with any device as a universal entity without going into the details of its specific operation, which greatly simplifies the extensibility of the system when adding new types of devices that simply need to fulfill this contract.

```
// The interface defines only a set of actions, abstracting
from the object type.
public interface IOnOffToggleable
{
    void TurnOn(bool isElectricityOn);
    void TurnOff();
    void ToggleState(bool isElectricityOn);
}

// Use in click handling:
if (device is IOnOffToggleable toggleable)
    toggleable.ToggleState(IsElectricityOn);
```

Abstraction is used through the implementation of the `IOnOffToggleable` interface for devices that support manual state switching. This makes it possible to handle mouse clicks on different objects, such as lamps, fans, or thermostats, using a single universal method, ignoring their class affiliation and focusing only on their common ability to be turned on or off.

```
public abstract class ClimateControlUnit : SmartDeviceBase,
IOnOffToggleable
{
    // Common implementation for all climate devices
    public void TurnOn(bool isElectricityOn)
    {
        if (isElectricityOn && CurrentState == DeviceState.Off)
        {
            CurrentState = DeviceState.Working;
        }
    }
}

// HeaterDevice does not duplicate TurnOn, but uses the parent
abstraction
public class HeaterDevice : ClimateControlUnit { ... }
```

Abstraction is used in the creation of an intermediate abstract class, `ClimateControlUnit`, which generalizes the logic of operation for the entire group of climate control devices. This allows to hide and centralize the basic power management mechanisms in one place, thanks to which specific classes of heaters or conditioners are freed from duplicating routine code and contain only temperature checking algorithms specific to them.

```
// Base class from which all devices inherit
public abstract class SmartDeviceBase : INotifyPropertyChanged
{
    public string Name { get; }
    public Point Position { get; set; }

    protected SmartDeviceBase(string name, string id, Room room,
Point position, Size size, MainWindow mw)
    {
        Name = name;
        Position = position;
        // ... initialization of other common fields
    }
}
```

USE OF INHERITANCE

Inheritance is used when declaring the abstract class `SmartDeviceBase` as the parent base for the entire hierarchy of smart home components. This avoids code duplication by defining common characteristics such as name and location in one place and ensures that these properties are automatically passed on to all descendant classes.

```
// WindowDevice is a child of SmartDeviceBase
public class WindowDevice : SmartDeviceBase
{
    // The constructor passes parameters to the base class
    public WindowDevice(string name, string id, Room room,
Point pos, Size size, MainWindow mw) : base(name, id, room,
pos, size, mw)
    {
        CurrentState = DeviceState.Working;
    }

    // Override the base class method
    public override void Interact(ToolType tool, bool
isElectricityOn, TimeSpan currentTime)
    {
        if (tool == ToolType.Hammer)
        {
            CurrentState = DeviceState.Off;
            AssociatedRoom.IsBreached = true;
        }
    }
}
```

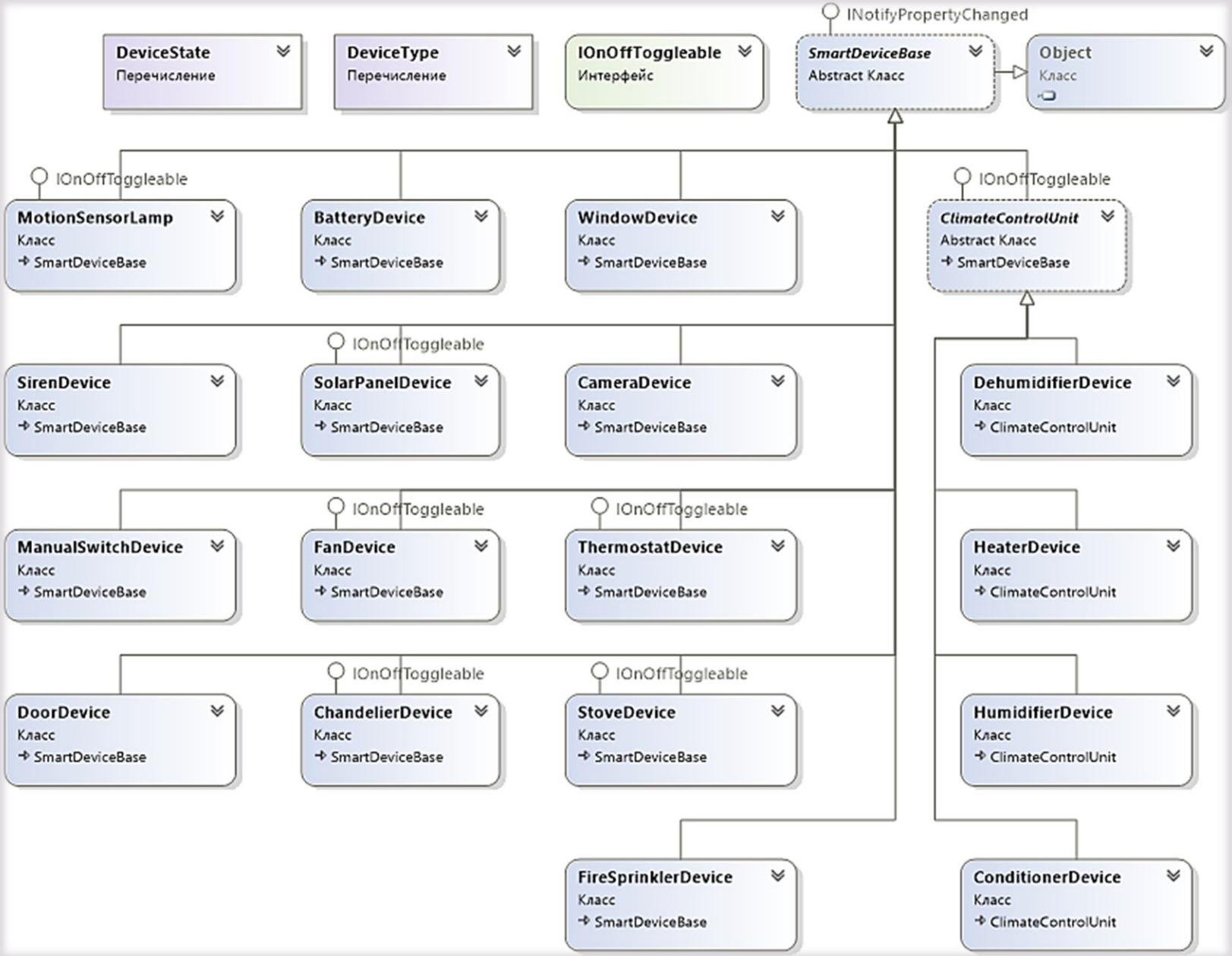
Inheritance is used, as example, in the `WindowDevice` class, which inherits the basic functionality from the parent class `SmartDeviceBase` through colon syntax. This allows to avoid rewriting common state saving mechanisms and only override the specific Interact method to implement a unique window response to the hammer tool.

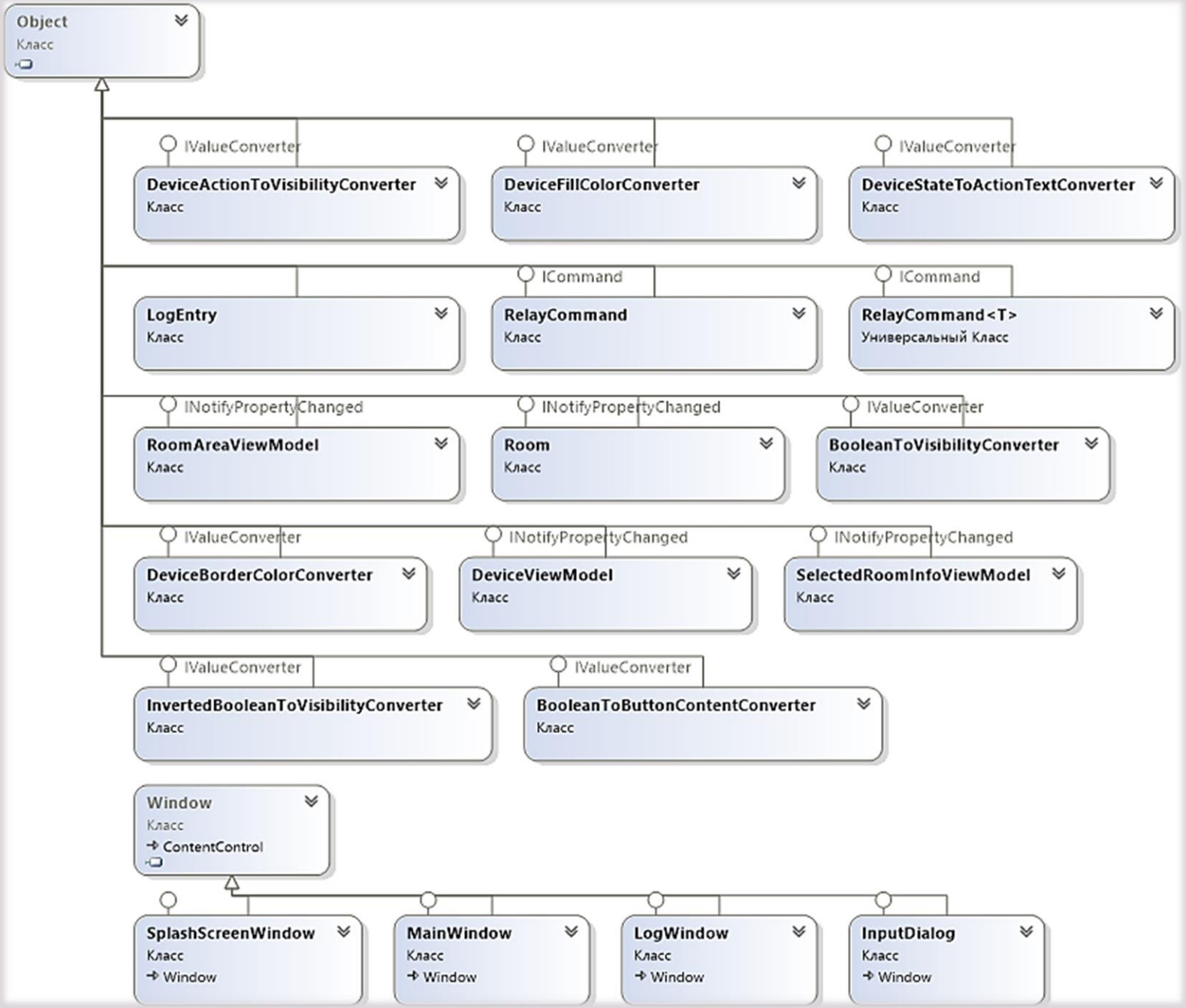
```
// Intermediate class inherits from SmartDeviceBase
public abstract class ClimateControlUnit : SmartDeviceBase,
IOnOffToggleable
{
    // Common logic for all climate control equipment
    public void TurnOn(bool isElectricityOn) { ... }
}

// HeaterDevice inherits from ClimateControlUnit, receiving
its methods
public class HeaterDevice : ClimateControlUnit
{
    public override void UpdateState(...)
    {
        // Only unique temperature checking logic here
        // TurnOn/TurnOff methods are taken from the parent
class
    }
}
```

Inheritance is used to create an intermediate abstraction layer through the `ClimateControlUnit` class, which combines a group of devices with similar behavior. This makes it possible to move the logic for switching power modes, which is common to both heaters and conditioners, to this intermediate class and significantly simplify the code of the end devices, leaving only unique temperature control algorithms in them.

CLASS DIAGRAMS




Object
Класс
IValueConverter
DeviceActionToVisibilityConverter
Класс
IValueConverter
DeviceFillColorConverter
Класс
IValueConverter
DeviceStateToActionTextConverter
Класс
LogEntry
Класс
ICommand
RelayCommand
Класс
ICommand
RelayCommand<T>
Универсальный Класс
INotifyPropertyChanged
RoomAreaViewModel
Класс
INotifyPropertyChanged
Room
Класс
IValueConverter
BooleanToVisibilityConverter
Класс
IValueConverter
DeviceBorderColorConverter
Класс
INotifyPropertyChanged
DeviceViewModel
Класс
INotifyPropertyChanged
SelectedRoomInfoViewModel
Класс
IValueConverter
InvertedBooleanToVisibilityConverter
Класс
IValueConverter
BooleanToButtonContentConverter
Класс
Window
Класс
ContentControl
SplashScreenWindow
Класс
Window
MainWindow
Класс
Window
LogWindow
Класс
Window
InputDialog
Класс
Window

```
// The _allDevices list contains objects of different types
(Camera, Heater, Siren)
foreach (var device in _allDevices)
{
    // Polymorphic call: the version of the method for the
specific object is executed
    // The controller does not know what kind of device it
is, but it knows that it can be updated
    device.UpdateState(CurrentSimTime, powerFlag,
device.AssociatedRoom);
}
```

USE OF POLYMORPHISM

Polymorphism is used in the main simulation timer when iterating over the `_allDevices` list, which stores references to the base type `SmartDeviceBase`. This allows the central controller to process different objects such as cameras, thermostats, and sprinklers in a single cycle by calling the common `UpdateState` method for them. Thanks to this approach, the system architecture becomes open for expansion, since adding a new device type does not require changes to the main program cycle code.

```
// Realization for solar panel: depends on time of day
public override void UpdateState(TimeSpan currentTime, ...)
{
    if (currentTime.Hours >= 8 && currentTime.Hours < 16)
        CurrentState = DeviceState.Working;
}

// Realization for the heater: depends on the ambient
temperature
public override void UpdateState(TimeSpan currentTime, ...)
{
    if (environment.CurrentTemperature < targetTemp)
        TurnOn(isElectricityOn);
}
```

Polymorphism is used directly in the realization of the abstract `UpdateState` method in descendant classes, where each device defines its own unique logic for responding to the passage of time. This allows the system to achieve complex component behavior using the same call command. In the example above, the solar panel reacts to the time interval of a day, while the heater ignores time and reacts only to the values of the environmental temperature sensors.

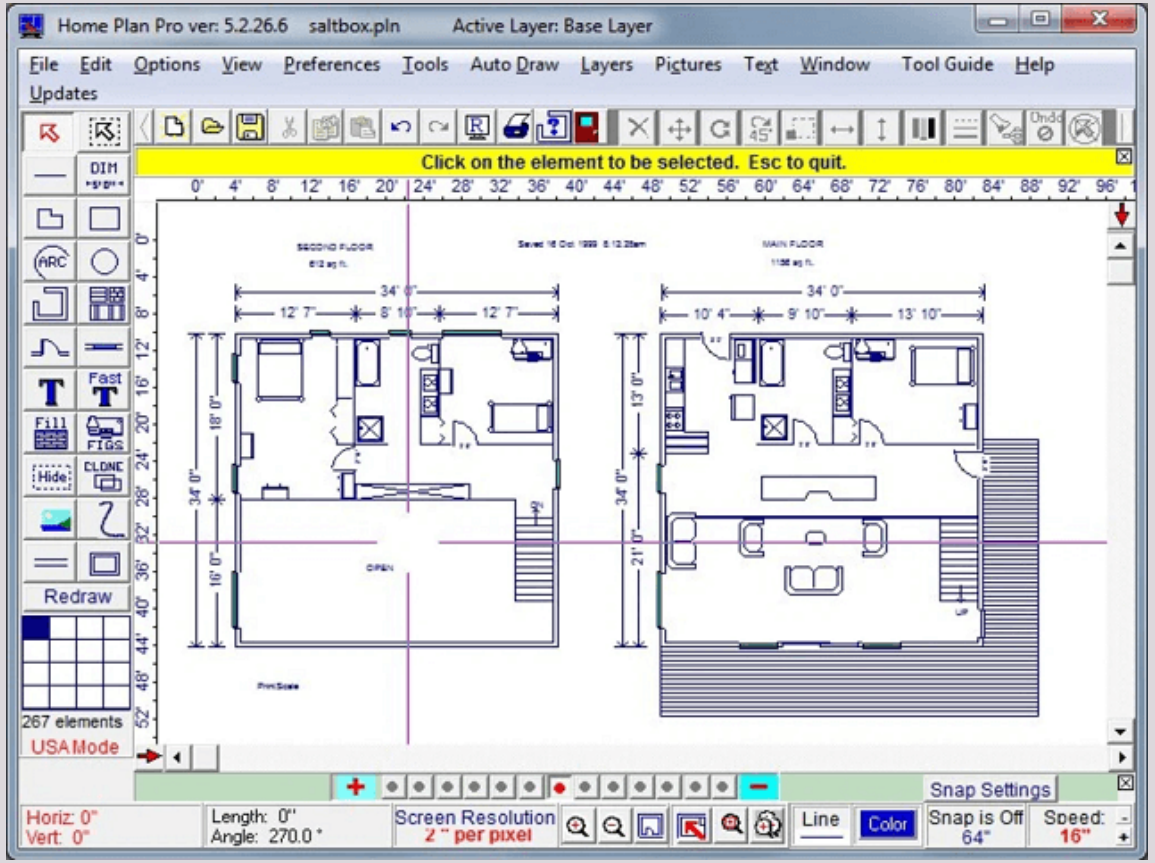
```
// Check if the device supports the toggle interface
if (device is IOnOffToggleable toggleable)
{
    // Call the interface method without knowing the actual
device type
    // For a lamp, this will turn on the light; for a fan, it
will turn on the rotation
    toggleable.ToggleState(IsElectricityOn);
}
```

Polymorphism is used in the `Device_MouseLeftButtonDown` mouse event handler by checking for membership in the `IOnOffToggleable` interface. This allows to realize a universal manual control mechanism for devices that are completely different in purpose, such as fans, chandeliers, or kitchen stoves. The system does not check the specific class of the object but works with its ability to be switched, which greatly simplifies the input processing code and reduces the number of if-else statements.

PRACTICAL VALUE AND PERSPECTIVE FOR FURTHER DEVELOPMENT

The practical value of the resulting system lies in its ability to model and debug smart home scenarios without the need for expensive physical equipment. This allows you to safely test the logic of device interaction and identify potential problems at the design stage, which significantly reduces the risks of actual system implementation.

The perspective for further development and improvement of the system is the integration of a house plan editor, which will allow users to independently create and change the topology of rooms for simulation. An additional direction is the introduction of scripting language support, which will allow the creation of advanced user automation scenarios without the need to recompile the program.



THE END